# GraphQL + GraphDB Case Study Report Structure

## 📘 Case Study Report

1️⃣ **Title Page :** Social Network Management using GraphQL and Neo4j Graph Database

**Student Name:** Ragini Singh

**Roll No :** 150096724023

**Cohort :** Jensen Huang

**Case Study No :** 134

**Semester:** IV Sprint 1

**Subject:** GraphQL / Graph Database

**College:** ITM Skills University

**Submission Date:** 14th March Saturday

**Github Repo Link :**

https://github.com/RaginiSingh2024/graphql-neo4j-social-netwok

**Documentation Link :**

https://docs.google.com/document/d/1vANSBU0GlCscDTKU6cM66O6FCCd7F7YxGnoXnlyrXpg/edit?usp=sharing

## 2 Introduction :

GraphQL is a modern API query language that allows clients to request exactly the data they need.
Graph databases store data as nodes and relationships rather than tables.

In this project, a social network system is implemented using GraphQL and Neo4j graph database. The system manages users, posts, comments, likes, and friend requests.

Neo4j is used as the graph database and GraphQL API is implemented using Strawberry GraphQL and FastAPI.

## 3 Problem Statement :

The goal of this case study is to design and implement a social network system using GraphQL and a graph database. The system should support user interactions such as creating posts, commenting, liking posts, and sending friend requests. The database structure should efficiently represent relationships between users and content.

## 4 — Abstract

This case study demonstrates the implementation of a **social network management system using GraphQL and Neo4j graph database**. Graph databases are designed to manage highly connected data efficiently.

In this project, users can create posts, comment on posts, like posts, and send friend requests. Neo4j is used as the graph database to represent users and their interactions through nodes and relationships. GraphQL is used as an API layer to query and manipulate the data.

The system demonstrates how graph databases provide efficient ways to manage complex relationships in applications such as social networks.

## 5 Tools and Technologies Used:

| Tool | Purpose |
|---|---|
| Neo4j | Graph Database |
| Cypher | Graph Query Language |
| GraphQL | API Query Language |
| Strawberry GraphQL | GraphQl Implementation |
| FastAPI | API Server |

| | |
|---|---|
| Python | Backend language |

## 6 Graph Data Model :

**Entities:**

- User
- Post
- Comment
- FriendRequest
- Like

**Relationships:**

- POSTED
- COMMENTED
- LIKED
- SENT_REQUEST
- ON

## 7. Neo4j Database Implementation

### Example 1 — Create User

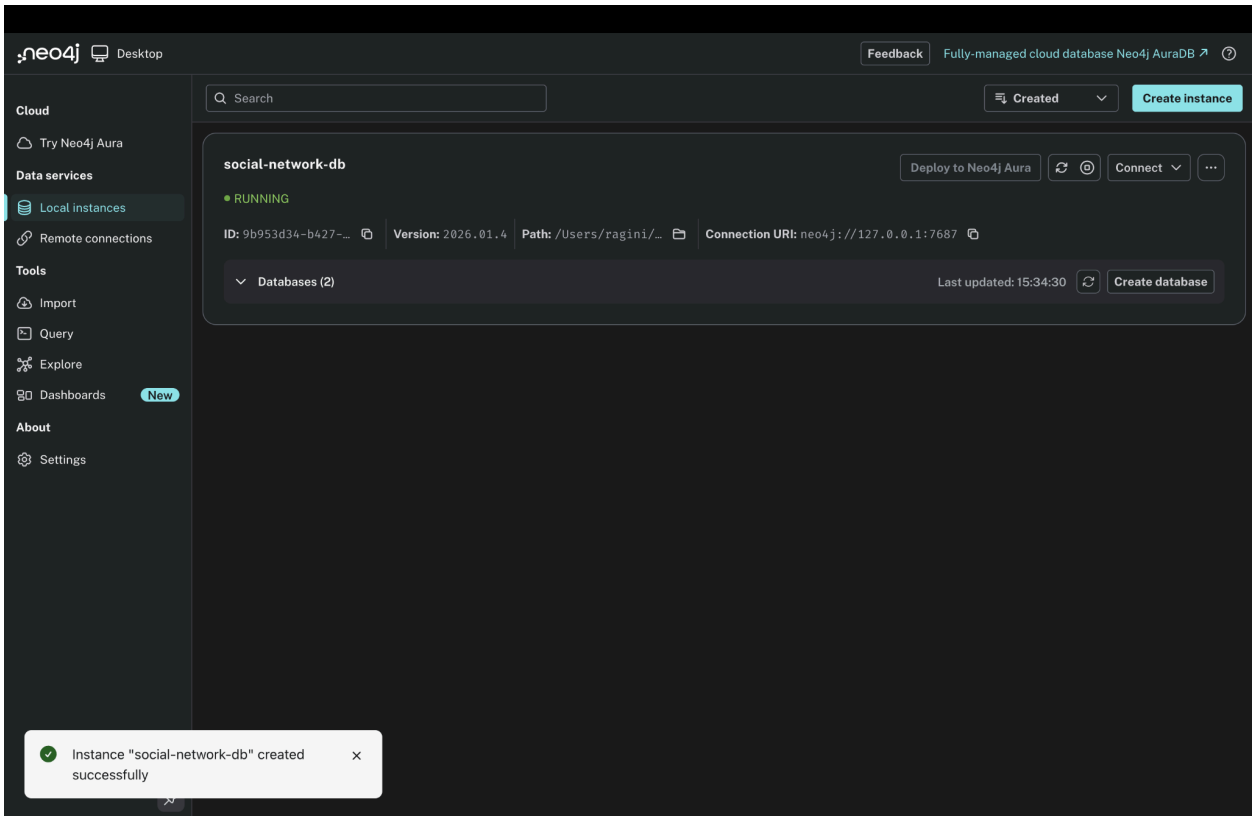
### Create User Node

To create a user node in the Neo4j graph database, the following Cypher query was executed.

```
CREATE (u:User {
```

id:1,
name:"Ragini",
email:"ragini@gmail.com",
bio:"CSE Student",
status:"active"
})

### 📷 Screenshot 1 — Neo4j Instance Running




**Caption: Figure 1: Neo4j Database Instance Running**

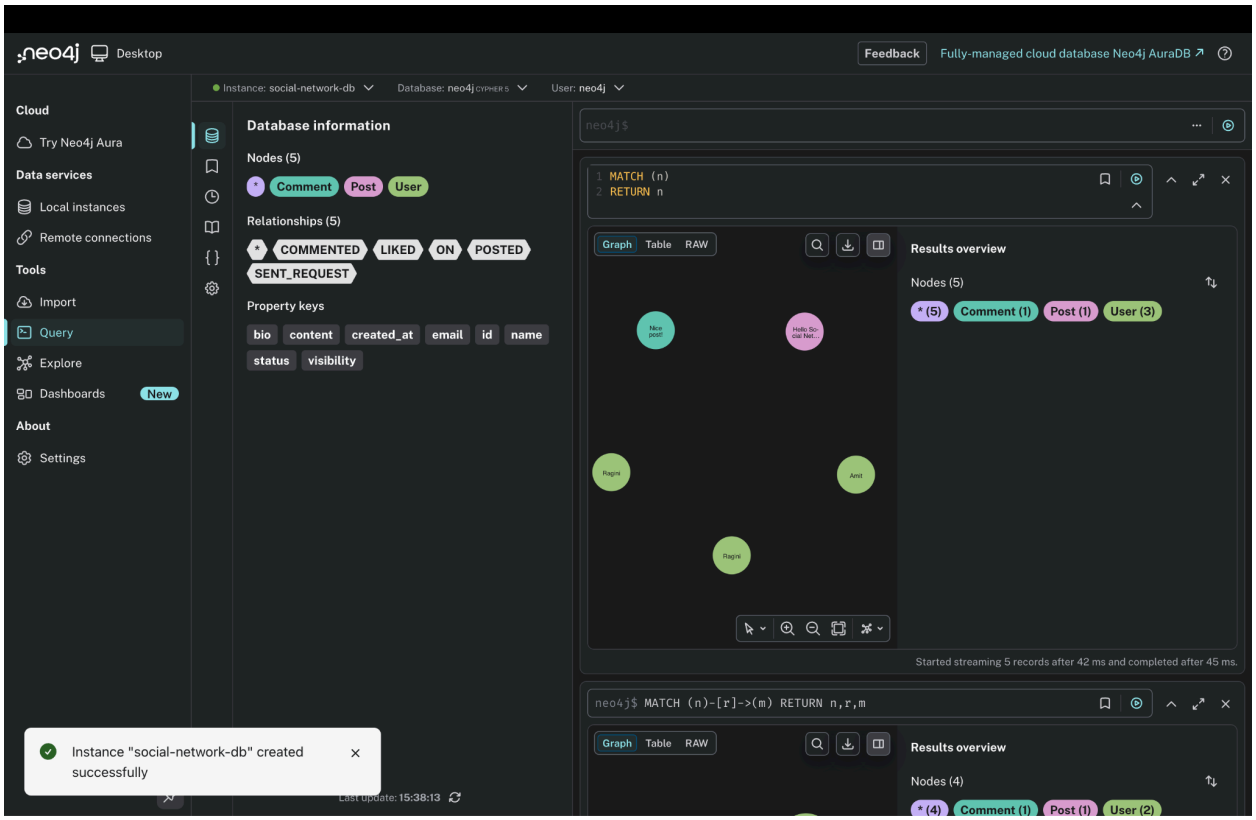
# Example 2 — Create Post

## Create Post Node

A post node is created and connected with the user using the POSTED relationship.

```
MATCH (u:User {id:1})
CREATE (p:Post {
id:101,
content:"Hello Social Network",
created_at:"2026-03-04",
visibility:"public"
})
CREATE (u)-[:POSTED]->(p)
```

---



**Caption: Figure 2: Post Node Created and Connected with User**

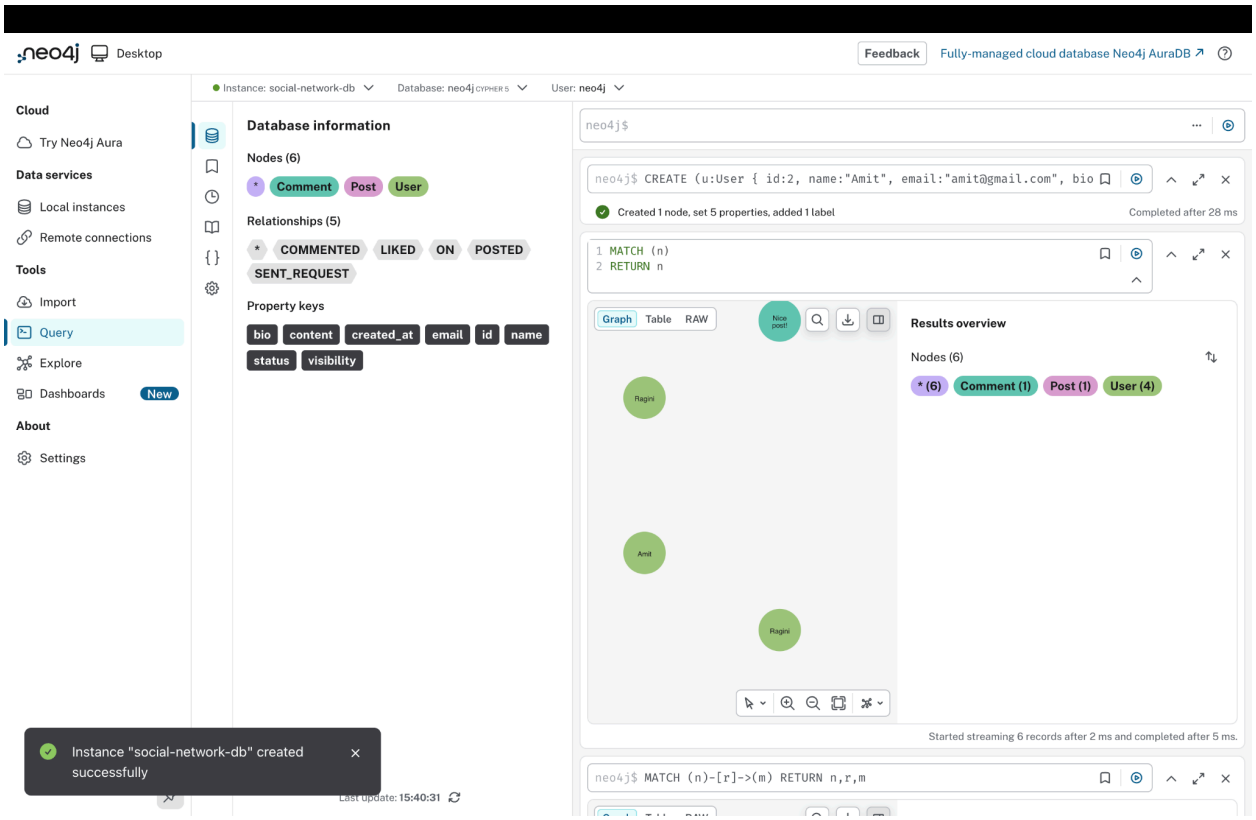
## Example 3 — Comment Relationship

**Create Comment**

A comment node is created and linked with the post using the COMMENTED and ON relationships.

```
MATCH (u:User {id:2}), (p:Post {id:101})
CREATE (c:Comment {
id:1,
content:"Nice post!"
})
```

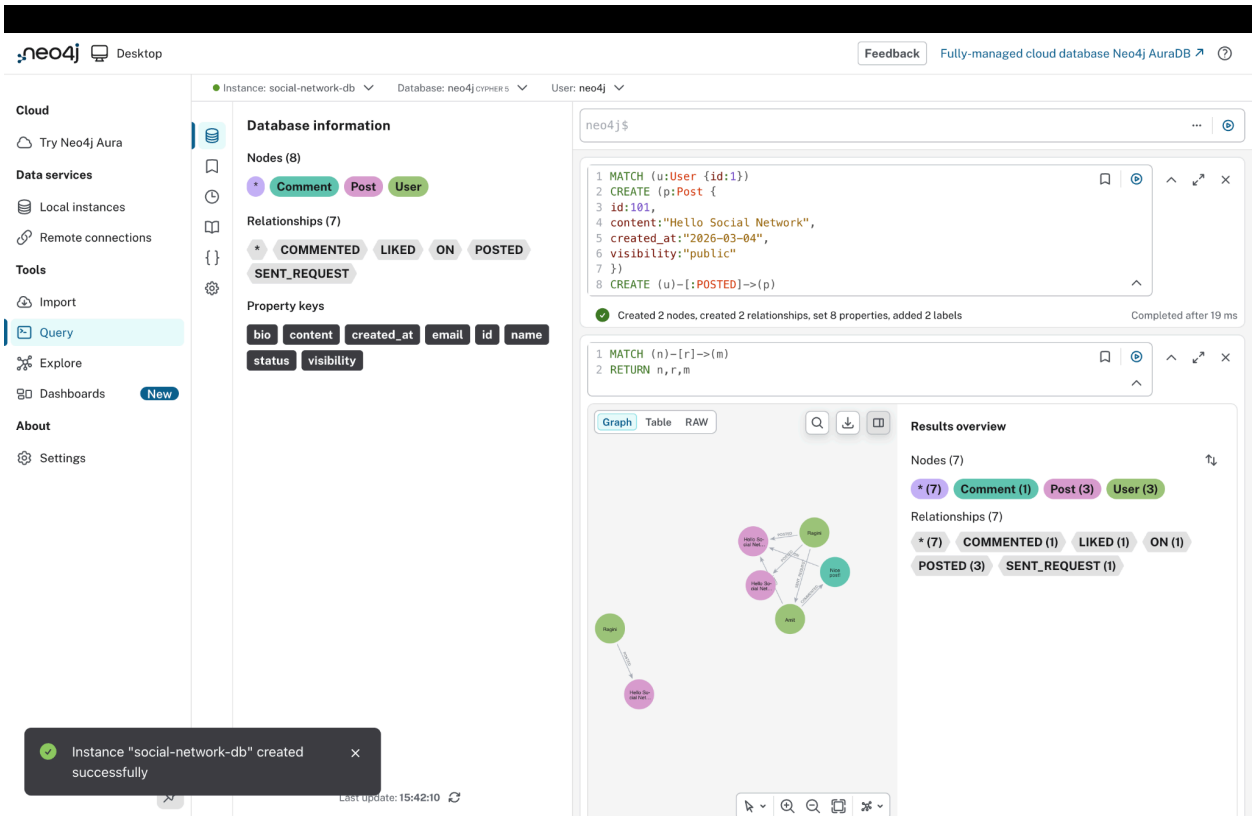
CREATE (u)-[:COMMENTED]->(c)
CREATE (c)-[:ON]->(p)




**Caption: Figure 3: Comment Relationship between User and Post**

# Example 4 — Like Relationship

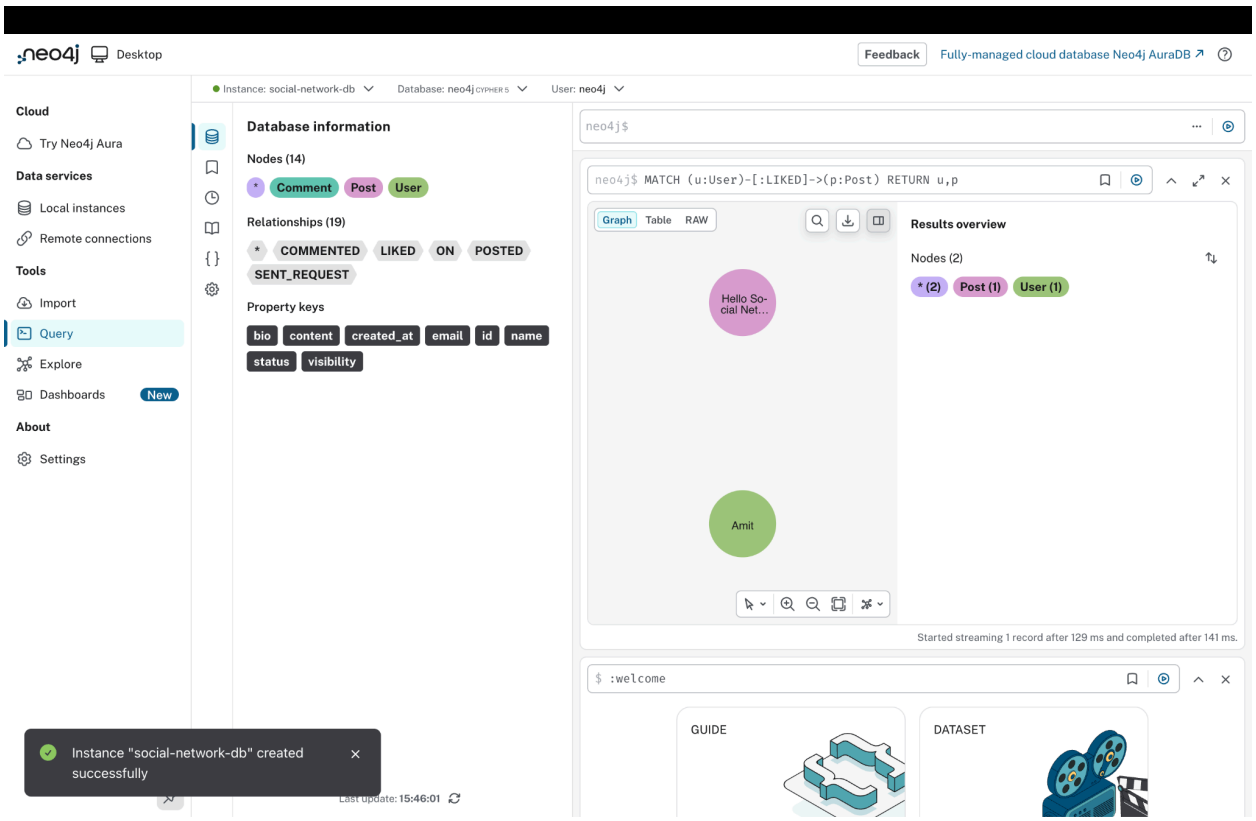
MATCH (u:User {id:2}), (p:Post {id:101})
MERGE (u)-[:LIKED]->(p)




**Caption: Figure 4: Like Relationship between User and Post**

# Example 5 — Friend Request

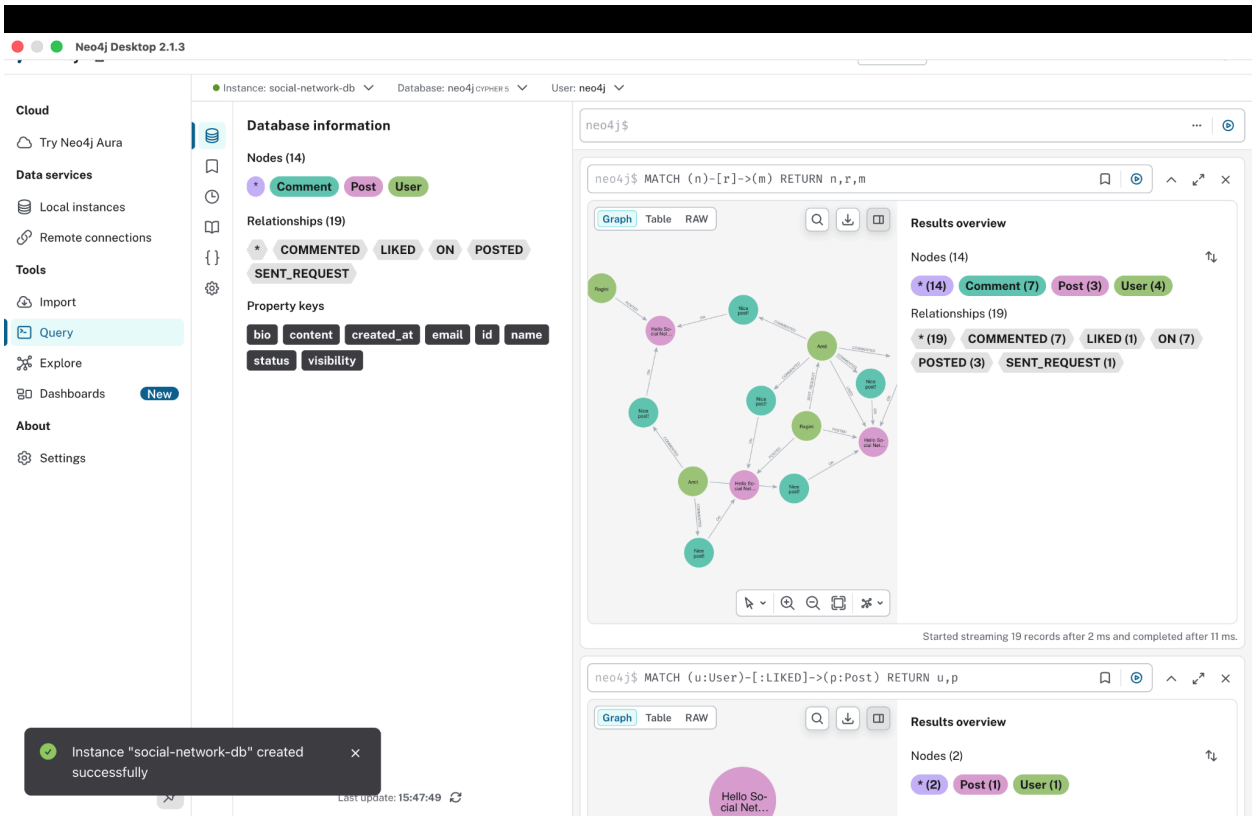
```
MATCH (a:User {id:1}), (b:User {id:2})
MERGE (a)-[r:SENT_REQUEST]->(b)
SET r.status="pending"
```



**Caption: Figure 5: Friend Request Relationship between Users**

# Example 6 — Graph Visualization

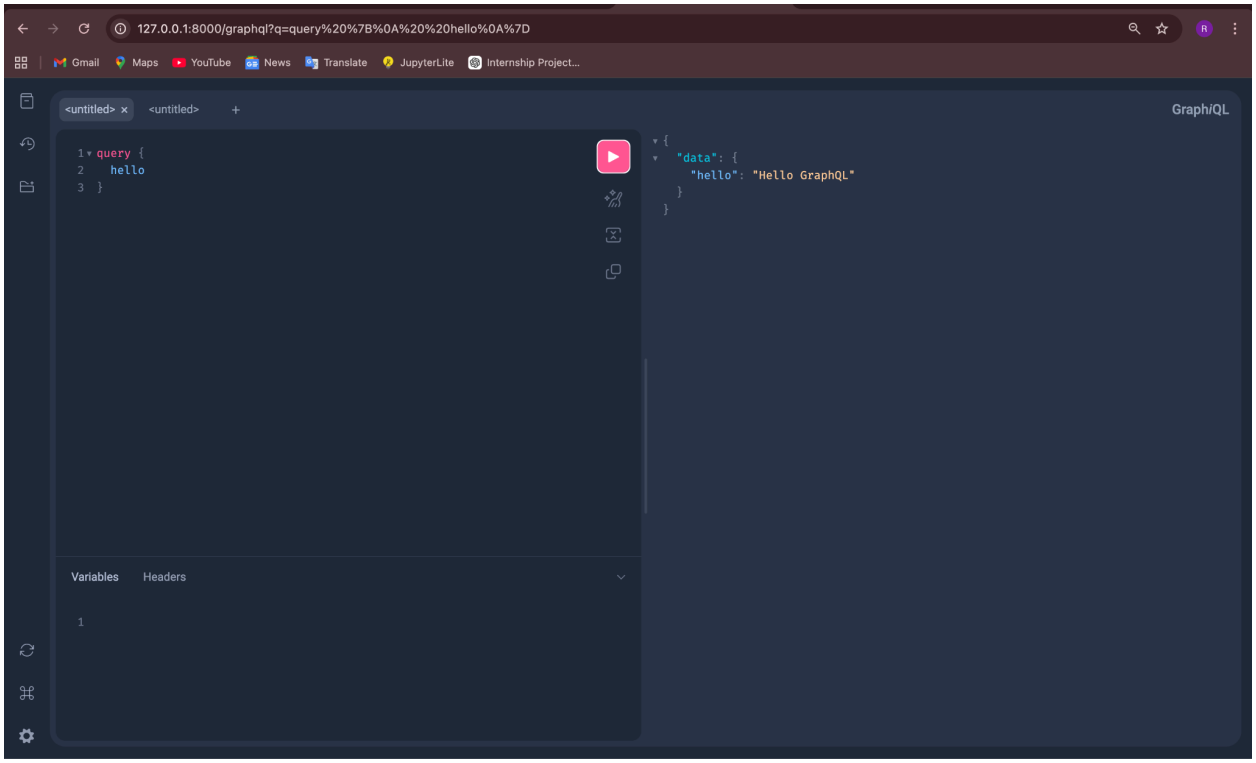
MATCH (n)-[r]->(m)
RETURN n,r,m




**Caption:  Figure 6: Complete Graph Visualization in Neo4j**

# Example 7 — GraphQL Query

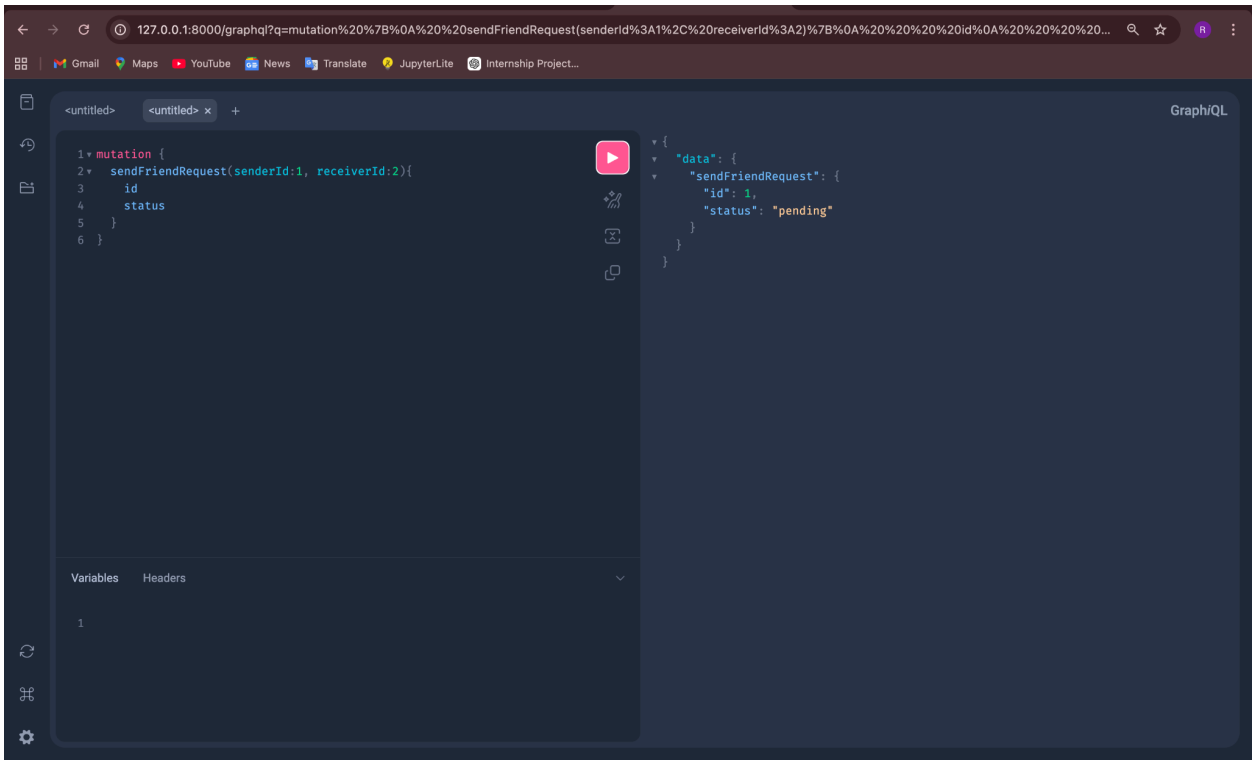
```
query {
 hello
}
```



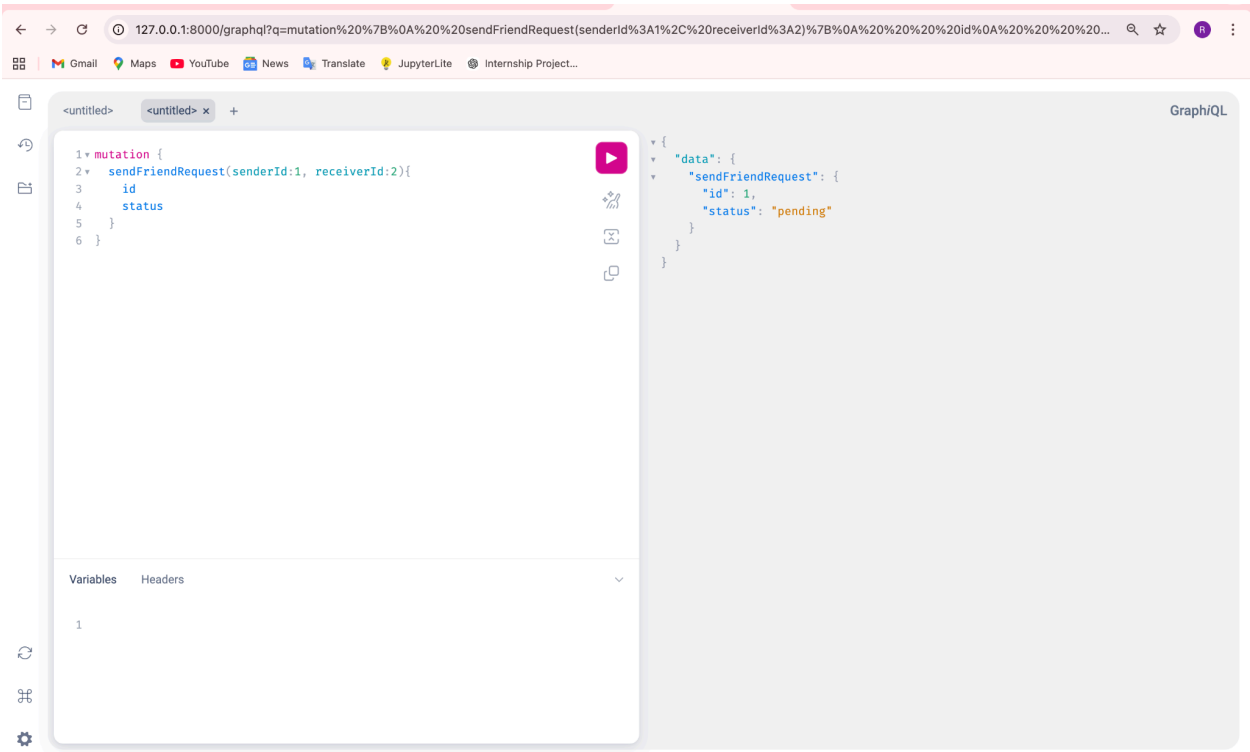

**Caption:Figure 7: GraphQL Query Execution**

# Example 8 — GraphQL Mutation

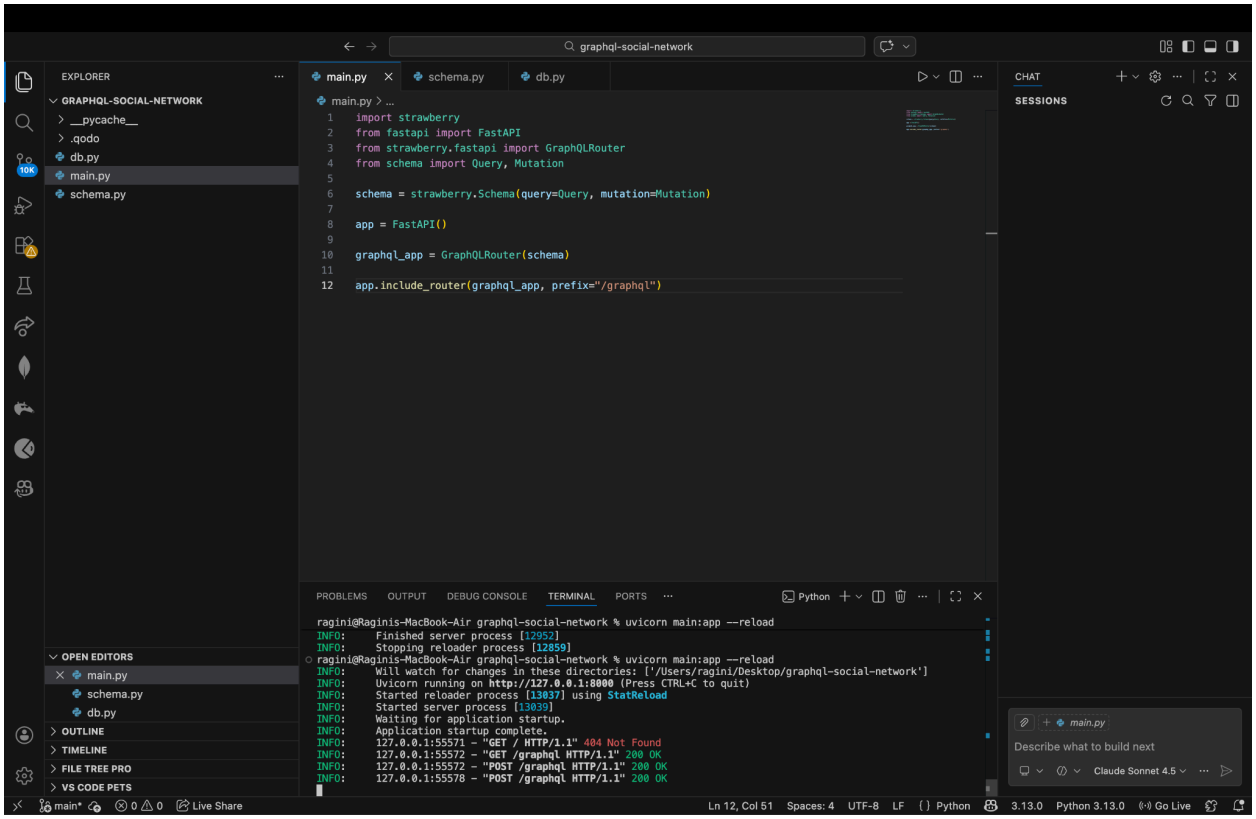
```
mutation {
 sendFriendRequest(senderId:1, receiverId:2){
  id
  status
 }
}
```



# Caption:Figure 8: GraphQL Mutation Execution



# Folder Structure :



## 📄 Conclusion

This case study demonstrates the implementation of a social network management system using GraphQL and Neo4j graph database. Graph databases are highly effective in managing connected data structures such as social networks.

In this project, users, posts, comments, likes, and friend requests were modeled as nodes and relationships in the Neo4j database. Cypher queries were used to create and manage these relationships.

Additionally, a GraphQL API was implemented using FastAPI and Strawberry GraphQL to interact with the graph database. GraphQL allows clients to request only the required data, making it efficient and flexible for modern applications.

Overall, this project highlights how GraphQL and graph databases can work together to efficiently manage complex relationships in applications like social networking platforms.

## 📄 Future Scope

Although the current system demonstrates the basic functionality of a social network using GraphQL and Neo4j, several enhancements can be implemented in the future.

The system can be extended by adding advanced features such as friend recommendation systems, real-time notifications, and social analytics. Graph algorithms provided by Neo4j can also be used to analyze user connections and suggest potential friends.

In addition, the system can be integrated with authentication and user management features to create a fully functional social networking platform.

**THANKYOU!!!!!!!**